Final Project: Q-Learning Strategy on High Frequency Trading

(Anonymized)

Content

0. Abstract

1. Keywords

2. Introduction

3. Data

4. Techniques and Theory Details in the Strategy

. Reinforcement Learning

. Q-Learning

. Monte Carlo Method and First-Visit Monte Carlo

. ε-greedy strategy

5. Implementation of our Algorithm

6. Result/Analysis

7. Citation and other Resources

0.Abstract

The reinforcement learning methods are applied to optimize and make the decision in our trading algorithm in our project. We use classic reinforcement learning algorithm, Q-learning and Monte Carlo techniques to evaluate the performance of the trading and update decision making of the trading based on the feedback of the previous trading status.

Moreover, first-visit Monte Carlo (FVMC) method is introduced when we are trying to estimate our Q-value by depending on the Sharpe ratio status of a day t. We also use different terms of Sharpe-ratio of the strategy as a state variable to test our strategy, and use an ε-greedy strategy to decide our action to the trading.

1.Keywords

Financial Prediction, Machine Learning, Reinforcement learning, Q-Learning, Sharpe Ratio, Monte Carlo, ε-greedy strategy

2.Introduction

In real world, machine learning is broadly applied into different industries, including finance, robotics, vision, medical treatment and so on. In this project, we applied a specific type of machine learning, reinforcement learning into our trading strategy. Reinforcement learning is a type of machine learning, and thereby also a branch of Artificial Intelligence. It allows machines and software agents to automatically determine the ideal behavior within a specific context, in order to maximize its performance. Reward feedbacks is required for the agent to learn its behavior, and it is known as the reinforcement signal. The behavior can be learnt once and for all, or keep on adapting as time goes by. With careful modeling of our strategy, some reinforcement learning algorithm can potentially converge to the global optimum, which is the ideal behavior that maximizes the reward. Like a human, the agents learn for themselves to achieve successful strategies that lead to the greatest long-term rewards, and learn actually similar to how human learns.

The reason we choose reinforcement learning for decision making in our trading strategy is because that it doesn't only learn from its past information on the market, but also the newly updated information feedback from the learning happened right before the current status, which we can potentially expect less overfitting than classical machine learning algorithms. AlphaGo which used deep reinforcement learning in its final phase to play games against itself by adapting and learning all the information from its past games directly in order to improve, and it does improve significantly. At this level, we are able to identify the patterns of the financial data much faster and learn from the trial and error (RL) without fear of overfitting.

Q-learning is a specific model-free reinforcement learning technique that we use in this project to train our data. This technique can be used to find an optimal action-selection policy for any given finite Markov decision process (MDP). It works by learning an action-free function that ultimately gives the expected utility of taking a given action in a given state and following the optimal policy thereafter. When such an action-value function is learned, the optimal policy can be constructed by simply selecting the action with the highest value in each state.

3.Data

Since we are going to focus on high frequency trading in our project, we are going to select the most liquid assets in the market. In addition, we would like to explore asset sectors with various volatilities. Later, we found Foreign Exchange market and ETFs interest three of us the most. Hence, we narrowed our focus to the two product sectors.

We did a little bit research on the data too. According to an article named 'How Forex Relates to the Equity Markets' on Nasdaq.com, "Increases or the prospect of increases in domestic interest rates will generally raise the value of the local currency while it will tend to depress domestic equity markets...during periods of sustained interest rate increases, equities markets can become unattractive and foreign exchange trading attractive." Based on his conclusion, we started to wonder whether the trend help us hedge risk in the case of Q-learning. Going further, If all the models are trained to be stable, will it still be necessary to hedge risk?

We started our data collection from a Bloomberg machine inside the FinMath Lab. We randomly selected 6 traded foreign currencies at 2 minutes per data point from 5/1/2017 to 5/5/2017 for data training. We also did the same to the ETF market. We not only ensured the data to be intact, but also the volume to be enough to support our simulate trading volume. Aside from that, we made sure that the assets are liquid enough for high frequency trading and allow our positions to enter and exit without any barriers.

The selected Foriegn Exchange currencies are: USDJPY, USDEUR, USDCAD, USDGBP, USDAUD, USDCHF The selected ETFs are: XLU, GLD, SMH, XBI, XOP, OIH

With that, we started our exploration.

In [1]:
```
# Initial the analysis environment
from RL_strategy import *
from IPython.display import Image
init_environment()
%matplotlib inline
```

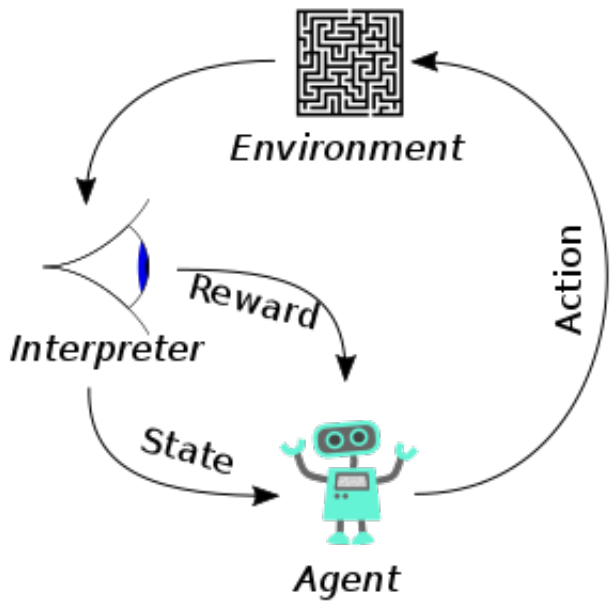
4.Techniques and Theory Details in the Strategy

Reinforcement Learning

The basic reinforcement learning model consists of:

1. a set of environment and agent states S;
2. a set of actions A of the agent;
3. policies of transitioning from states to actions;
4. rules that determine the scalar immediate reward of a transition; and
5. rules that describe what the agent observes.

The rules are often stochastic. A vivid graph to describe such process is provided by Wikipedia:



A reinforcement learning agent interacts with its environment in discrete times steps. At each time t, the agent receives an observation O_t, which typically includes the reward r_t. It then chooses an action a_t from the set of actions available, which is subsequently sent to the environment. The environment moves to a new state s_{t+1} and the reward r_{t+1} associated with the transition (s_t, a_t, s_{t+1}) is determined. The goal of a reinforcement learning agent is to collect as much reward as possible.

The reinforcement learning problem requires clever exploration mechanisms. Randomly selecting actions without reference to an estimated probability distribution is known to give rise to very poor performance. In practice people resort to simple exploration methods. One such method is ε-greedy strategy. When the agent chooses the action that it believes has the best long-term effect with probability $1 - \varepsilon$, and it chooses an action uniformly at random, otherwise. Here $0 < \varepsilon < 1$ is a tuning parameter, which is sometimes changed, either according to a fixed schedule, or adaptively based on some heuristics.

In the reinforcement learning problem, a critical part is the value function approach. Value function approaches attempt to find a policy that maximizes the return by maintaining a set of estimates of expected returns for some policy. Although state-values suffices to define

optimality, it will prove to be useful to define action-values. Given a state S, an action A and a policy π, the action value of the pair under (S, A) is defined by

$$Q^{\pi}(S, A) = E[R|S, A, \pi]$$

where, now R stands for the random return associated with first taking action A in state S and following π, thereafter. There are two well-known approaches to compute the optimal action value function, value iteration and policy iteration, both of which compute a sequence of functions $Q_k(k = 0, 1, 2, \dots)$ which converge to Q^*.

Q-Learning

Q-learning is a specific algorithm of reinforcement learning. The problem consists of an agent, states S and a set of actions per state A. by performing an action $a \in A$, the agent can move from one state to another. Executing an action in a specific state provides the agent with a reward (a numerical score). The goal of the agent is to maximize its total reward. It does this by learning which action is optimal for each state. The optimal action for a specific state has the highest long-term reward. This reward is a weighted sum of the expected values of the rewards of all future steps starting from current state, where the weight for a step from a state Δt steps into the future is calculated as $\gamma^{\Delta t}$, where γ is a number between 0 and 1 $(0 < \gamma < 1)$ called the discount factor and trades off the importance of sooner versus later rewards.

The quantity of a state-action combination (S, A) is always defined as a map:

$$Q : S \times A \to \mathbb{R}$$

Q takes an arbitrary value initially chosen by the designer. Then each time the agent selects an action, and observes a reward and a new state that my depend on both previous state and the selected action, Q is updated. Thus the core idea of Q-learning is a value iteration update process, where a next quantity is based on the old value of the quantity and the correction made based on new information learnt:

$$Q(s_t, a_t) \leftarrow \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha_t}_{\text{learning rate}} \cdot \left(\overbrace{\underbrace{r_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}}}^{\text{learned value}} - \underbrace{Q(s_{\text{old}}}_{\text{old}} \right.$$

where in this equation, r_{t+1} is the reward observed after performing a_t in s_t , and where α_t is the learning rate between 0 and 1 $(0 < \alpha_t < 1)$.

Overall, reinforcement learning is a critical way to solve sequential decision making problems, which is exactly what we want to do with trading strategy. At each time step t, the agent receives a state s_t and selects an action a_t from the space of the action A, following the policy $\pi(a_t|s_t)$, here $\pi(a_t|s_t)$ is the policy that follows the agent's behavior, and mapping from state s_t to actions a_t, receives a scalar reward r_t, and transitions to the next state s_{t+1} for reward function $R(S, A)$ and state transition probability $P(s_{t+1}|s_t, a_t)$ respectively. In an episodic problem, such procedure continues to happen until the agent arrives a terminal state and then it restarts. The return

$$R_t = \sum_{i=1}^{\infty} \gamma^k r_{t+k}$$

is the accumulated discounted reward with discount factor {\gamma} between 0 and 1. The agent aims to maximize the expectation of such long term return from each state. A value function is used to measure how good is a specific state-action pair (S, A). The defined function $Q^{\pi}(S, A) = E[R|S, A, \pi]$ is the expected return for selecting action a_t in state a_t and following the policy π.

The core idea in reinforcement learning and Q-learning is called temporal difference (TD). Temporal difference is the idea that learns value function $Q(s_t, a_t)$ directly from the experience (old value) with TD error in a model-free online way via bootstrapping method. The update rule is also discussed above, we again state below:

$$V(s_t) \leftarrow V(s_t) + \alpha[r_t + \gamma V(s_{t+1}) - V(S_t)]$$

Here α is the learning rate between 0 and 1, and the part after the learning rate: $r_t + \gamma V(s_{t+1}) - V(S_t)$ is called TD error. Similarly, Q-learning learns action value function, with the similar update rule:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma max_{\,a}\, Q(s_{t+1}, a) - Q(S_t, a_t)]$$

The algorithm above is called TD(0) and Q(0) process, with one step return. We also have multi-step return variants. When we talk about TD(1) and Q(1) return, it is exactly Monte Carlo approach.

Monte Carlo Method and First-Visit Monte Carlo

The Monte Carlo methods learn from compete sample returns, and it is only defined for episodic tasks and update after each episode (not step-by-step). Monte Carlo is useful in reinforcement learning and our trading strategy is because that it learn directly from

experience. The general MC methods applied in the learning process can be stated in a graph like below:

The simplest Monte Carlo methods can be used in an algorithm that mimics policy iteration. The Monte Carlo methods are used in the policy evaluation step. In the step, given a stationary, deterministic policy π, the goal is to compute the function values $Q^{\pi}(S, A)$ for all state-action pairs (S, A). Assuming that the Markov decision process (MDP) is finite and in fact a table representing the action-values fits into the memory, then the estimate of the value of a given state-action pair (S, A) can be computed by simply averaging the sampled returns which originated from (S, A) over time. Given enough time, this procedure can thus construct a precise estimate Q of the action value function Q^{π}.

In reinforcement learning using Monte Carlo policy evaluation, our goal is to learn $V^{pi}(s)$ (similarly what we want to learn is $Q(s_t, a_t)$ specifically in Q-learning), given some number of episodes under π contain s. The idea of our MC strategy is to average returns observed after visits to s, it can be expressed in a graph below,

The potential methods that can be applied here are first-visit MC and every-visit MC. In first-visit Monte Carlo methods, we average returns only for the first time s is visited in an episode, while in every-visit Monte Carlo methods, we average returns for every time s is visited in an episode.

The pseudo code is presented here to make the idea clear:

The use of Monte Carlo methods in our reinforcement learning is that both first-visit Monte Carlo and every-visit Monte Carlo methods are asymptotically convergent. In our trading strategy, we use first-visit Monte Carlo method for simplicity and faster computation.

ε-greedy strategy

The ε-greedy method is often used in sequential decision problems because the method is quite simple, and has obvious generalization for the sequential decision problem. At each specific time point, or say round $t = 1, 2, \ldots$, the ε-greedy method selects the arm with the highest empirical mean with probability $1 - \varepsilon$, and selects a random arm with probability ε. In other words, to give a specific form of the algorithm, given initial empirical means $\hat{\mu}_1(0), \hat{\mu}_2(0), \ldots, \hat{\mu}_K(0)$

$$p_i(t+1) = \begin{cases} 1 - \varepsilon + \varepsilon/k & \text{if } i = argmax_{j=1,2,\ldots K}\, \hat{\mu}_j(t) \\ \varepsilon/k & \text{otherwise} \end{cases}$$

In order to make sure, in our Q-learning trading strategy, by using the ε-strategy, we are really improving on every step, I can offer a simple proof as below:

Let π' be the ε-policy in our trading Q-learning strategy,

$$\begin{aligned}
Q^{\pi}(s, \pi'(s)) &= \Sigma_a \pi'(s,a) \cdot Q^{\pi}(s,a) \\
&= \frac{\varepsilon}{|A(s)|} \Sigma_a Q^{\pi}(s,a) + (1-\varepsilon) max_a Q^{\pi}(s,a) \\
&\geq \frac{\varepsilon}{|A(s)|} \Sigma_a Q^{\pi}(s,a) + (1-\varepsilon) \Sigma_a \frac{\pi(s,a) - \frac{\varepsilon}{|A(s)|}}{1-\varepsilon} \cdot Q^{\pi}(s,a) \\
&= \frac{\varepsilon}{|A(s)|} \Sigma_a Q^{\pi}(s,a) - \frac{\varepsilon}{|A(s)|} \Sigma_a Q^{\pi}(s,a) + \Sigma_a \pi(s,a) Q^{\pi}(s,a) \\
&= V^{\pi}(s)
\end{aligned}$$

From the equations and calculations above, we can figure out that $V^{\pi'}(s) \geq V^{\pi}(s)$ $\forall s \in S$, or equivalently $\pi' \geq \pi$. Again, here π is the ε-greedy policy in our trading Q-learning strategy.

Now we see, by applying ε-greedy policy to our trading strategy with Q-learning, we have improvement on every step, except when the best policy has been found among the ε-greedy policies. We achieve the best policy only among the ε-greedy policies, and on the other hand, we have eliminated the assumption of exploring starts.

The pseudo code for the details of the ε-greedy policies can be stated in a way below:

One more point for us to notice is that, when ε in our algorithm is a constant, only a linear bound on the expected regret can be achieved. For situations that ε decreases with time, Cesa-Bianchi and Fisher (1998) proved poly-logarithmic bounds for variants of the algorithms.

5.Implementation of our Algorithm

In our process of implementation, we applied the reinforcement learning (Q-learning) to the trading strategies in FX market and some most liquid ETFs.

We use short-term and long-term Sharpe-ratio of the our previous positions to describe the current state, and choose the log return of the given length of our next position as our reward of Q-learning.

Since the limitation of computational abilities for normal laptops, instead of fitting a function for the Q-value, we use some Monte Carlo techniques for simplicity and easy computations.

In each round of training, the model has the following actions:

1. If it is the first round of training, set the Q-value matrix and the in-round reward matrix as zero; or the Q-value matrix is given by previous round and the in-round reward matrix is kept used.
2. At the beginning of the round, it will hold long position until there are enough actions to calculate the short-term and long-term Sharpe Ratio.
3. As there are Sharpe Ratios, the state can be described, then the next action of position can be decided by the Q-value matrix along with ε-greedy method, and the in-round reward matrix will be updated by the reward of this action using the update rule of Q-Learning.
4. At the end of the round, there is a renewed in-round reward matrix, and the new Q-value matrix is the weighted mean of the old Q-value matrix and the in-round reward matrix.

Loop the procedure above until the in-sample return converge and a fitted Q-value matrix is given, it usually takes 1000 to 2000 rounds.

The model training function is as following:

strategy_train(data_train_=fx_train.loc[:, 'USDJPY'], states_init=6, window_short_init=10, window_long_init=20, ssr_ratio=150, epsilon_init=0.4, rounds_init=1200, discount_init=0.5, learning_rate_init=0.1, hold_init=5)

As for the parameters of this funtion, ***data_train*** is the prices of the asset we use to train the model, ***states_init*** is the number of classes the Sharpe Ratios are separated into, ***window_short_init*** is the length of short-term previous strategy's returns, ***window_long_init*** is the length of long-term previous strategy's returns, "ssr_ratio" is the scale parameter to properly separate the Sharpe Ratios, "epsilon_init" is the value of ε we use while training, ***rounds_init*** is the number of loops we run to make the in-sample return convergent, ***discount_init*** and ***learning_rate_init*** are the γ and α introduced in Section 4, ***hold_init*** is the length of period we hold a position.

While traning the model, our goal is to find the highest in-sample convergent return, the following figure shows an example:

From the figure, first we can find that all returns are convergent, and Parameter Set C produces the best in-sample return, so we will choose the model trained with Parameter Set C to trade in the coming trading hours.

Once we have the Q-value matrix (i.e. decision matrix), we can use this matrix to conduct trading.

At the beginning of our trading, we will hold long positions untli we can use Sharpe Ratios to describe the state and use decision matrix to make decisions of position.

The testing function is as following:

strategy_test(model_, data_test_, fund, stop_loss=0.01, trading_cost_init=None)

As for the parameters of this funtion, ***model_*** is the model(strategy) given by the training function, ***data_test_*** is the prices of asset in our trading hours, ***fund*** is the size of each position, ***stop_loss*** is the stop loss line (1% as default) of each position, ***trading_cost_init*** is the transaction cost of each trade.

the return of this function is ***[pnl, period_decision_test, real_position_test]*** as ***pnl*** is the list of each pnl, ***period_decision_test*** is the list of decisions given by our strategy, and the ***real_position_test*** is the list of decisions considering stop loss.

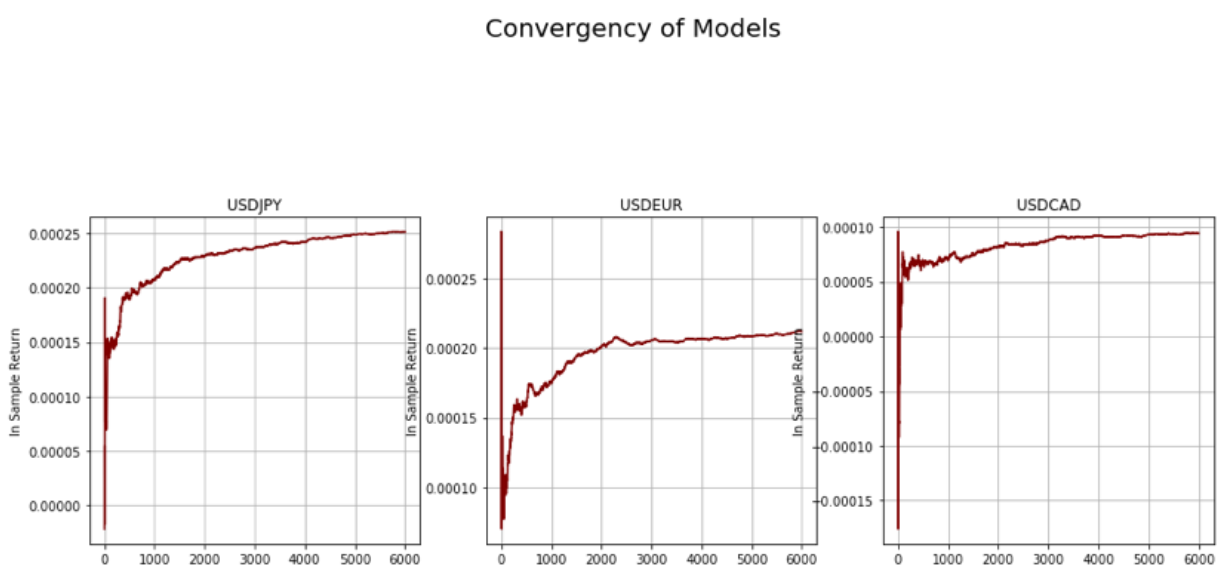
6.Result/Analysis

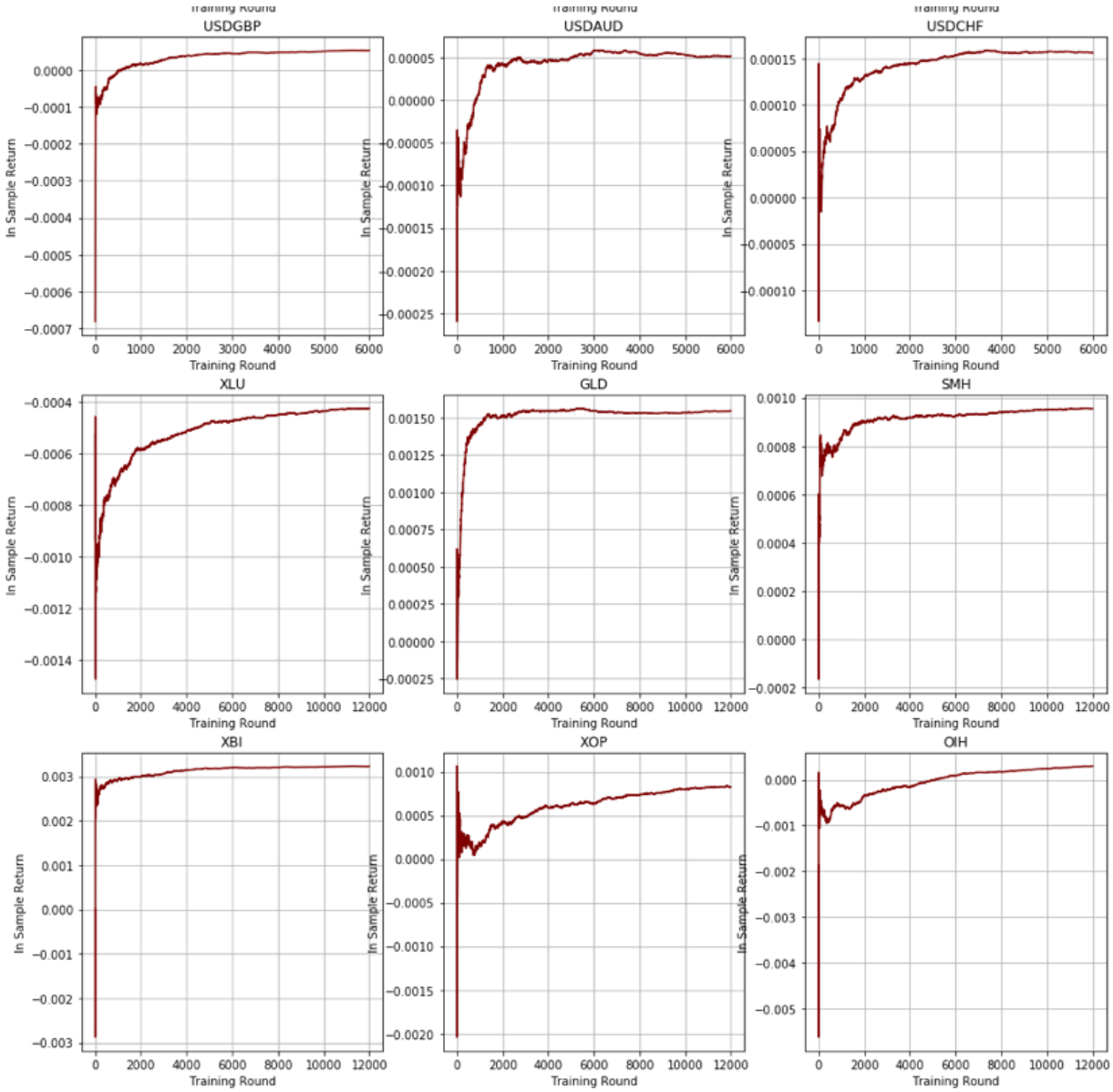
In [2]:
```
# prepare the data to use
train_start = '2017-05-01'
train_end = '2017-05-05'
trade_start = '2017-05-08 08:30:00'
trade_end = '2017-05-08 15:00:00'
[etf_train, etf_trade, fx_train, fx_trade] = data_prepare(train_sta
```

In [95]:
```
# model training
jpy_model = strategy_train(data_train_=fx_train.loc[:, 'USDJPY'], s
eur_model = strategy_train(data_train_=fx_train.loc[:, 'USDEUR'], s
cad_model = strategy_train(data_train_=fx_train.loc[:, 'USDCAD'], s
gbp_model = strategy_train(data_train_=fx_train.loc[:, 'USDGBP'], s
aud_model = strategy_train(data_train_=fx_train.loc[:, 'USDAUD'], s
chf_model = strategy_train(data_train_=fx_train.loc[:, 'USDCHF'], s
xlu_model = strategy_train(data_train_=etf_train.loc[:, 'XLU'], sta
gld_model = strategy_train(data_train_=etf_train.loc[:, 'GLD'], sta
smh_model = strategy_train(data_train_=etf_train.loc[:, 'SMH'], sta
xbi_model = strategy_train(data_train_=etf_train.loc[:, 'XBI'], sta
xop_model = strategy_train(data_train_=etf_train.loc[:, 'XOP'], sta
oih_model = strategy_train(data_train_=etf_train.loc[:, 'OIH'], sta

model_list = [jpy_model, eur_model, cad_model, gbp_model, aud_model
```

In [150]:
```
# show the convergency of each model's in-sample return
plot_convergence(model_list)
```



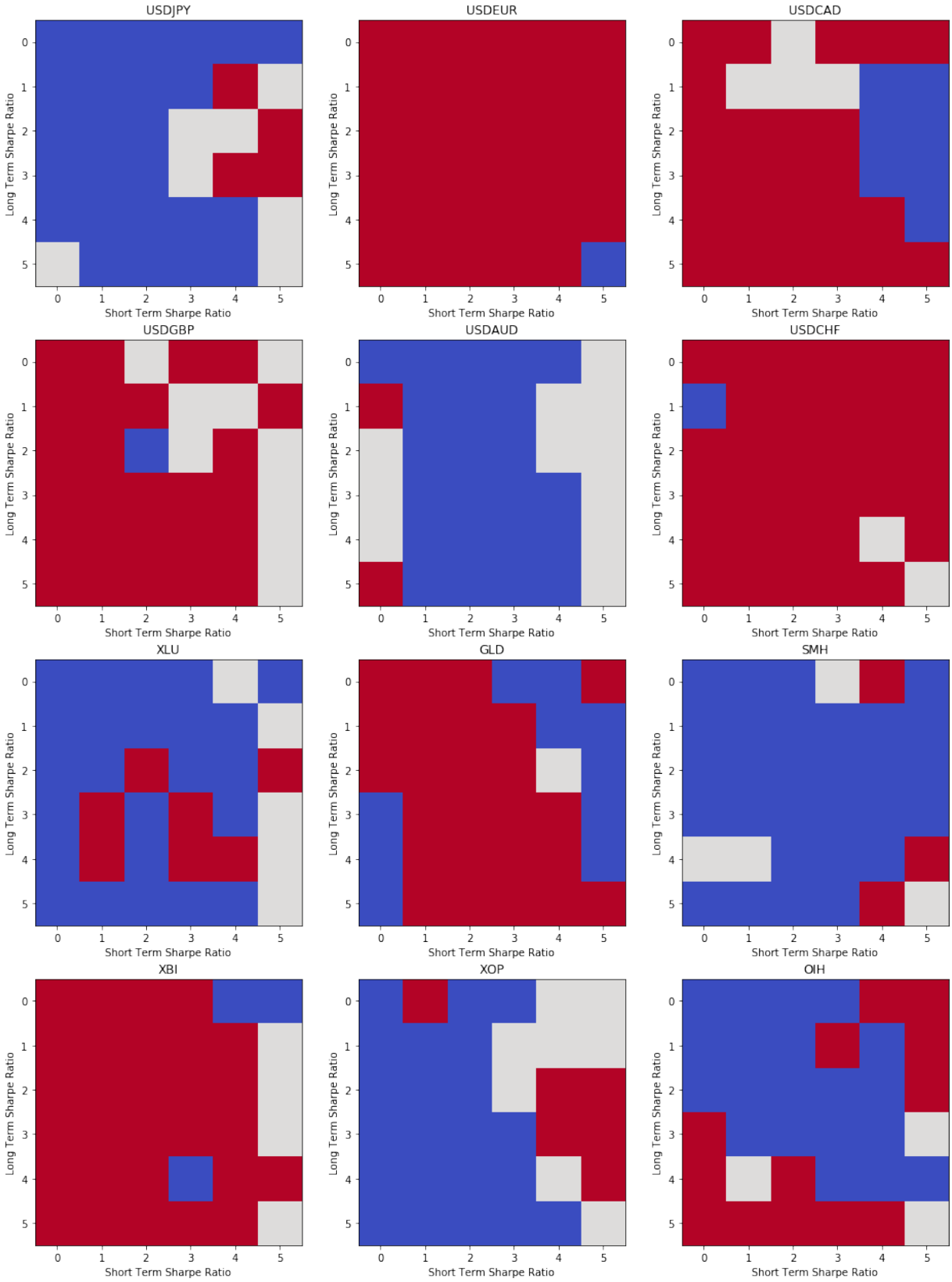


Analysis

As we have seen from the plots above, the Foreign Exchange currencies and the liquid ETFs are all performing convergently under our algorithm. This is exactly proving the fact we obtained from the theoretical part of the algorithm about the convergency of our algorithm. More importantly, it means all 12 models are stable enough and they are ready to use.

In [9]:
```
# show the decision matrices of each model
plot_decisions(model_list)
```

Decision Matrices

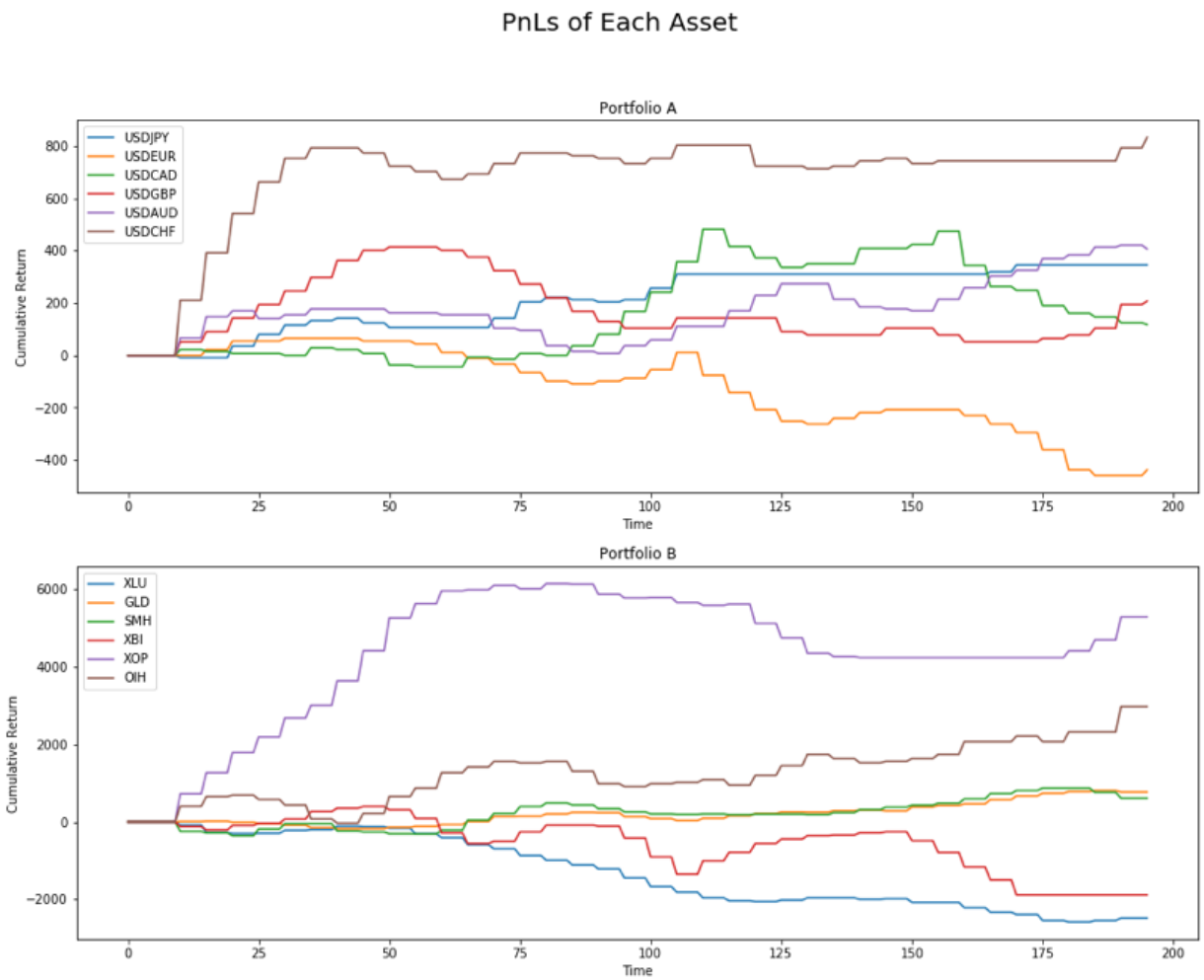

USDJPY
USDEUR
USDCAD
USDGBP
USDAUD
USDCHF
XLU
GLD
SMH
XBI
XOP
OIH
Long Term Sharpe Ratio
Short Term Sharpe Ratio
0 1 2 3 4 5

Analysis

Decision Matrix Analysis is the simplest form of Multiple Criteria Decision Analysis (MCDA), also known as Multiple Criteria Decision Aid or Multiple Criteria Decision Management (MCDM). In our model, we make decisions on our trading based on both the short term Sharpe Ratio and Long term Sharpe Ratio. In the decision matrix above, red represents for short, grey represents for doing nothing and blue reporesents for long. As we keep training our data, the decision matrix is converging to a stable status.

In [61]:
```
# test the models in our trading hours
result_list = pnl_result(model_list, fx_trade, etf_trade)
```

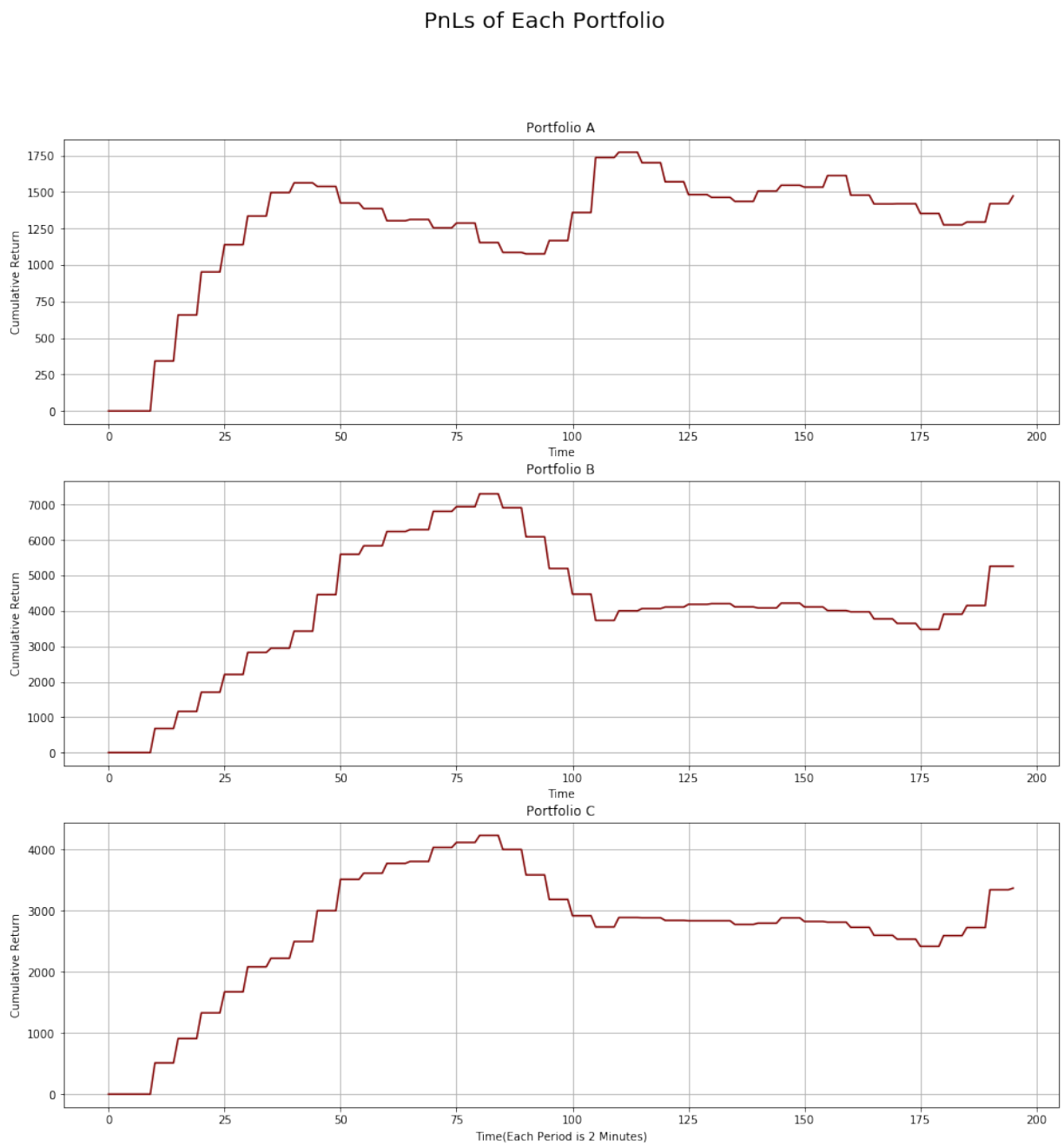
In [118]:
```
pnls_plot(result_list)
```



Analysis

To implement the algorithm to the real trading strategy, we chose 6 Foreign Exchange currencies and 6 liquid ETFs to train our algorithm. We trained each asset seperately first, and obtained the cumulative PnL plots shown above. As we can see from the previous outcome, all 12 models are trained to be stabel and ready to use. However, when we apply the model to trade, we found, the return of each individual asset can be positive or negative. Training model being stable doesn't garantee a positive return. Hence, we realized the importance to hedge through constructing portfolios.

In [143]:
```
# plot the pnl of each portfolio
[pnl_fx, pnl_etf, pnl_combined] = pnl_portfolio_plot(result_list)
```

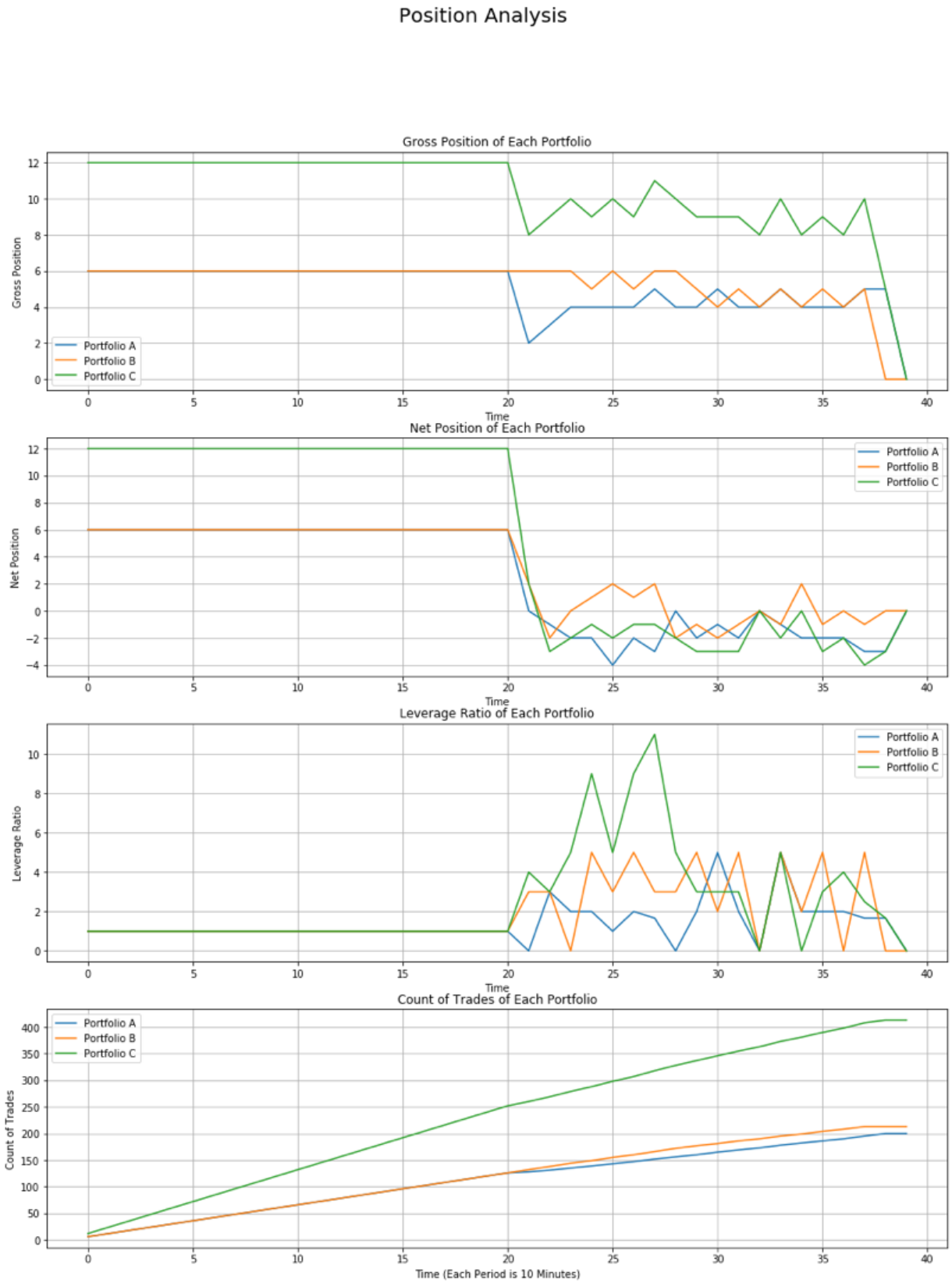


Analysis

We constructed three portfolios. Portfolio A is a combination of the 6 foreign exchange products. Portfolio B is a combination of 6 ETFs. Portfolio C is a combination of all 12 products. We allocated 1 million dollars to each portfolio. Hence, each asset in A and B will have 100,000 dollars trading limit, and each asset in C will have 50,000 dollars trading limit.

If we look close to the PnL, we found, Portfolio A generates the least amount of PnL among the three, with Portfolio B generates the most. Portfolio C stays in the middle.

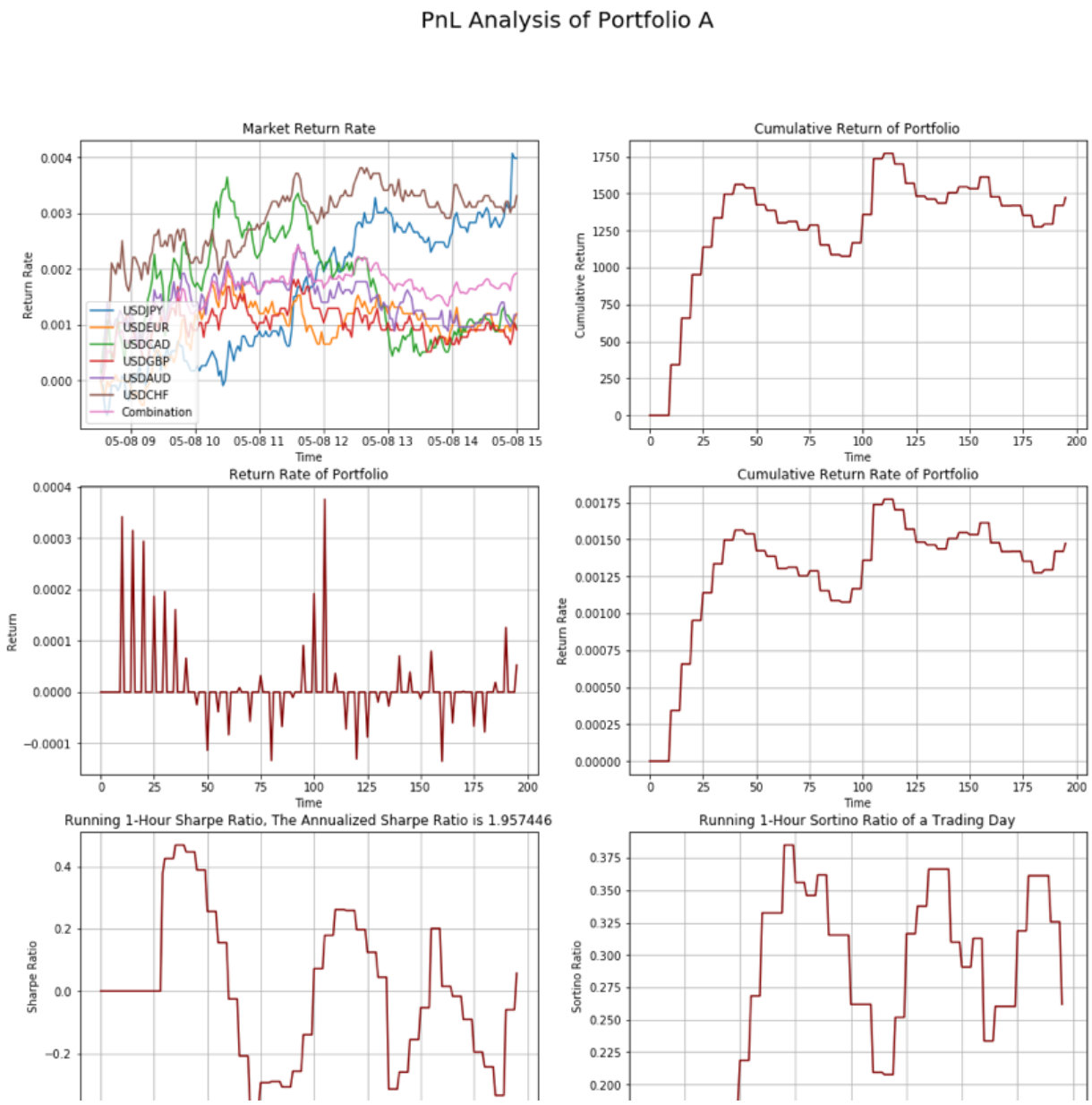
In [120]:
```
# analyse the position changes of each portfolio
position_analysis(result_list)
```



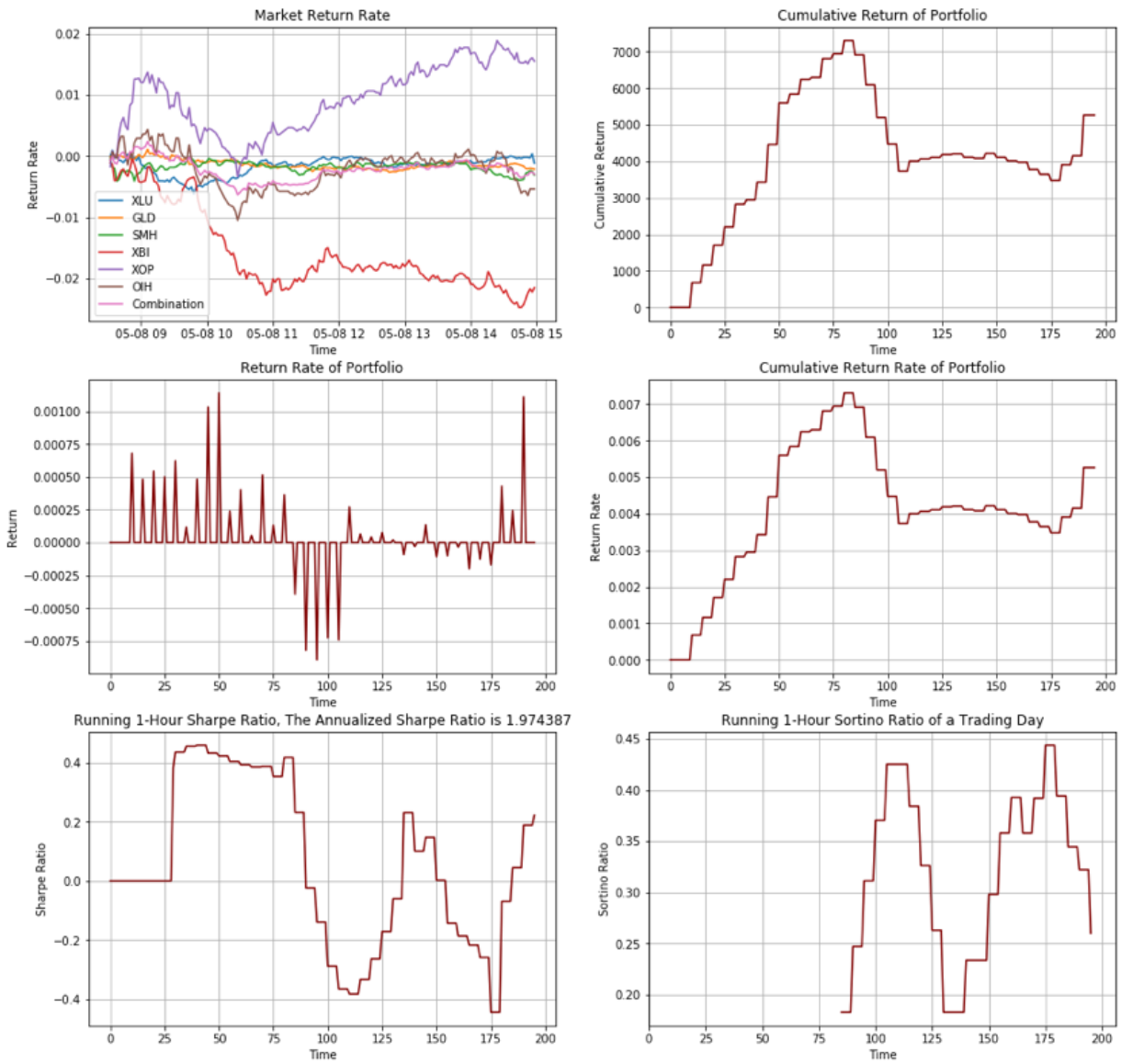
Anaylsis

We exercise our trade once every 10 minutes, i.e. we re-balance our portfolio once every 10 minutes. The horizons were caused by the special algorithms of our model. We assumed the position to be 'long' until the model gets enough information to make rational decisions. We also plotted the position and leverage over trading time for your reference. And, it is easy to see from those graphs that we hold more than 5 assets on hand at least some time period.

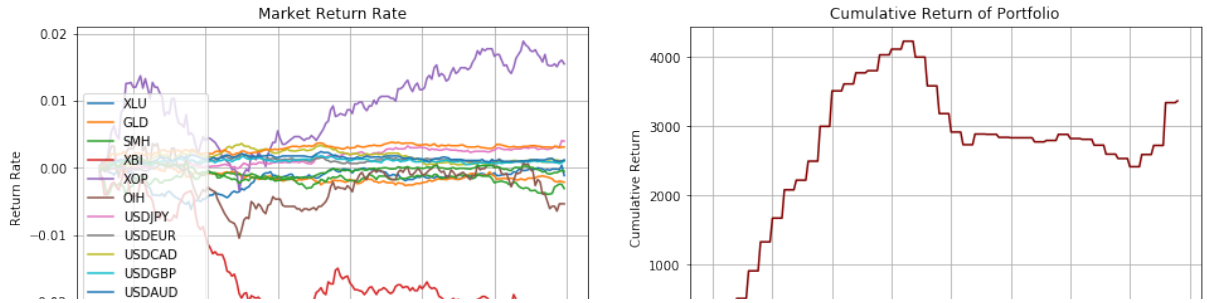
In [145]:
```
# analyse the pnl of each portfolio
pnl_analysis(result_list[0:6], fx_trade, 'Portfolio A', 1000000.0)
pnl_analysis(result_list[6:12], etf_trade, 'Portfolio B', 1000000.0
pnl_analysis(result_list, etf_trade.join(fx_trade), 'Portfolio C',
```

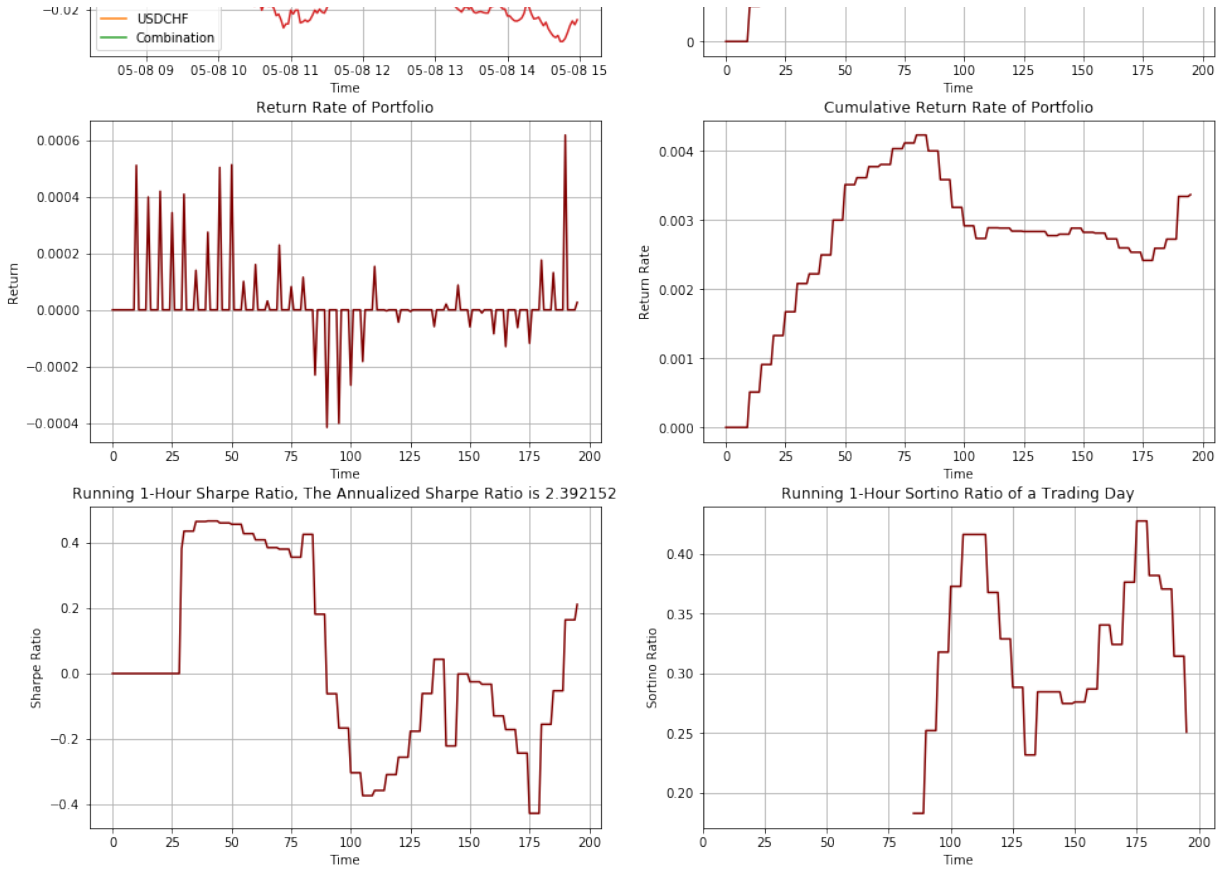


PnL Analysis of Portfolio B



PnL Analysis of Portfolio C





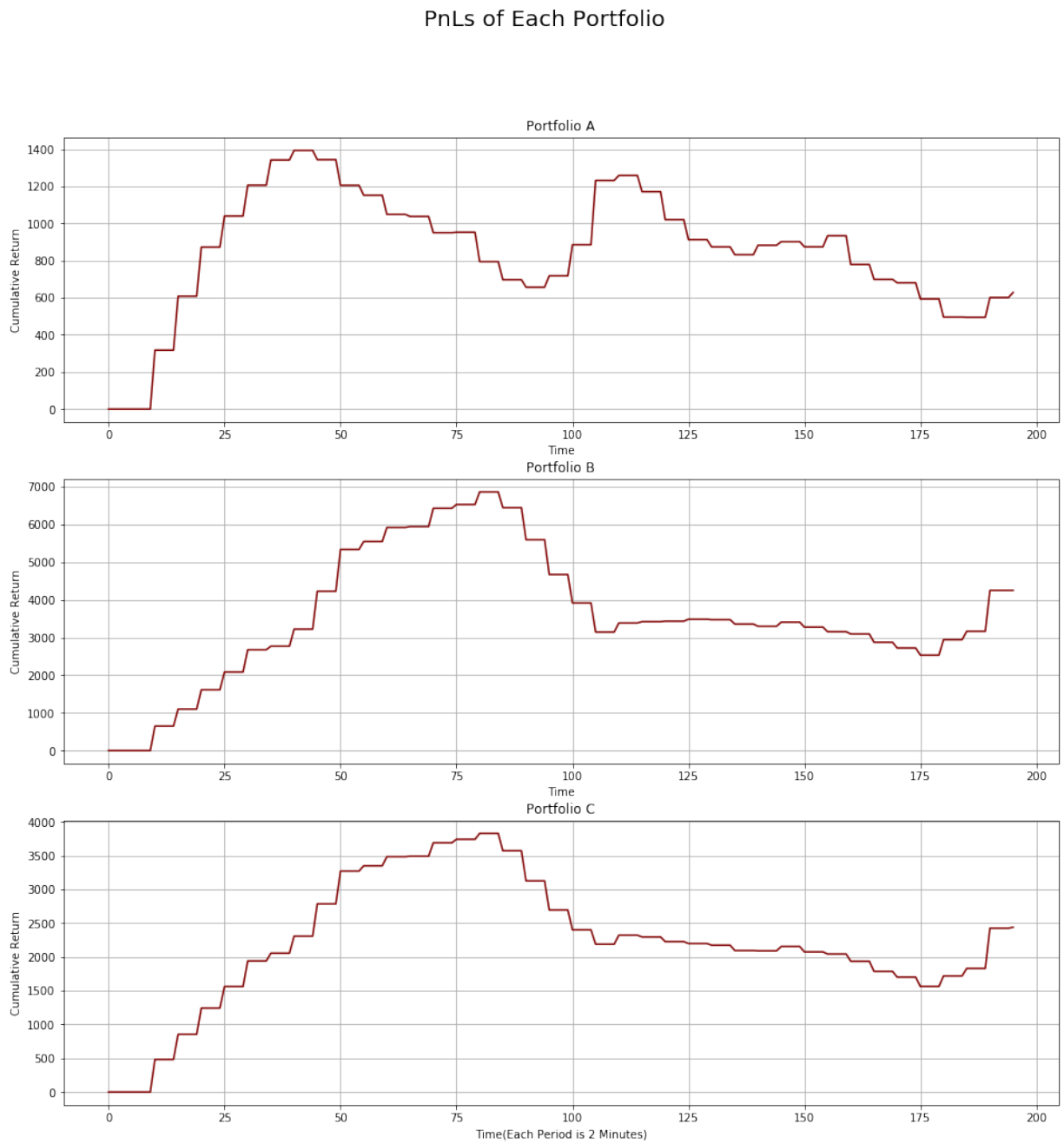
Analysis

As we saw from previous graphs, we have concluded that Portfolio B is the most profitable with Portfolio B being the least profitable. However, sophasiticated analyst would know that the best investment is not only represented by high returns. Investors also care about risks. Hence, we plotted annulized Sharpe ratio to compare. Surprisingly, we found, although Portfolio B has the highest return, Portfolio C has the highest Sharpe ratio with medium return. Prior research had proved that the Porfolio with the highest Sharpe ratio is the most attractive to invest. And rememeber, Porfolio C is the most diversified portfolio among three with assets across two carefully chosen sectors and 12 assets in total. Sortino ratios, which measures the loss, performs similarly across three portfolios.

In [148]:
```
# Value at Risk
print('FX:')
VaR(result_list[0:6])
print('ETF:')
VaR(result_list[6:12])
print('Combined:')
VaR(result_list, is_combined=True)
```

```
FX:
           0.1%          1%         5%        10%
VaR  180.910808  134.332293  92.777591  70.624933
ETF:
           0.1%          1%         5%         10%
VaR  640.121169  475.251961  328.16503  249.753531
Combined:
           0.1%          1%          5%         10%
VaR  335.155747  248.060425  170.358932  128.936554
```

In [146]:
```
# with transaction cost
result_tc_list = pnl_result(model_list, fx_trade, etf_trade, transa
[pnl_tc_fx, pnl_tc_etf, pnl_tc_combined] = pnl_portfolio_plot(resul
```



Analysis

Altough we assumed that we are market makers and our transaction cost can be rebated from the exchange. However, the PnL with transaction costs should considered to allow market takers to use this model. With a fixed 5 dollar cost per transaction, the general return trends did not change and we proved that all three portfolios can still be profitable.

Conclusion

First, from this model, we proved that Q-Learning models are effective with First-Visit Monte Carlo and ε-greedy methodologies. With the same set parameters, all 12 datasets can be trained to be stable and convergent. The only difference is that Foreign Exchange requires 1200 data points to converge while ETFs needs 2400 to converge. This difference may caused by different volatilities in the original data.

Second, although the models are stable enough to produce convergent results, we can still get negative returns for some assets. This requires us to build diversified porfolio to hedge the positions. And from our simulated trading result, the portfolios are profitable with decent Sharpe ratios.

Last but not least, our model still has a lot of room to improve. For example, more data points can be trained in order to gain a better and more stable model. A stronger computer can be employed at the same time to improve the fitting power. With that, we will be able to get a improved Q-Learning model.

7.Citation and other resources

[1] Xin Du, Jinjian Zhai, Koupin Lv, *Algorithm Trading using Q-Learning and Rucurrent Reinforcement Learning*

[2] Michael Kearns, Yuriy Nevmyvaka, *Machine Learning for Market Microstructure and High Frequency Trading*

[3] John Moody and Matthew Saffell, *Reinforcement Learning for Trading*

[4] Jae Won Lee, Jonghun Park,*Member,IEEE*, Jangmin O, Jongwoo Lee and Euyseok Hong, *A Multiagent Approach to Q-Learning for Daily Stock Trading*

[5] Yuxi Li, *Deep Reinforcement Learning: An Overview*

[6] Volodymyr Kuleshov, Doina Precup, *Algorithms for the multi-armed bandit problem*

[7] https://www.wikipedia.org/ (https://www.wikipedia.org/)

[8] http://reinforcementlearning.ai-depot.com/ (http://reinforcementlearning.ai-depot.com/)